

netmon operator runbook

netmon operator runbook

How to deploy, run, verify, and read the passive network monitor on any Linux host.

1. Requirements

- Linux (uses `AF_PACKET` raw sockets; macOS/Windows not supported)
- Python ≥ 3.13 and `uv` (`curl -LsSf https://astral.sh/uv/install.sh | sh`)
- `tcpdump` binary — only needed if you pass `--bpf` (scapy shells out to it to compile the filter)
- Root or `CAP_NET_RAW` (section 3)

2. Install

```
# copy the project to the target host, e.g.:  
rsync -a netmon.py pyproject.toml README.md RUNBOOK.md host:/opt/  
      netmon/  
ssh host  
cd /opt/netmon  
uv sync --no-dev
```

Without `uv`: `python3.13 -m venv .venv && .venv/bin/pip install scapy pydantic structlog cryptography`, then substitute `.venv/bin/python` `netmon.py` for `uv run netmon.py` everywhere below.

After moving or renaming the project directory, re-run

`uv sync`. The venv's console-script shebangs (`.venv/bin/mypy`, etc.) are absolute paths baked in at creation; a move leaves them pointing at the old path, so `.venv/bin/<tool>` fails with `bad interpreter` until `uv sync` regenerates them. `uv run ...` is

unaffected (it re-resolves the interpreter), but a dir move also invalidates the `setcap` grant from section 3B — re-apply it.

3. Capture privileges — pick one

A. sudo per run (simplest, nothing persistent):

```
sudo $(command -v uv) run netmon.py
```

`sudo uv` alone usually fails — root's PATH lacks `~/.local/bin`, hence `$(command -v uv)`.

B. Grant the capability once (for unattended use and running without sudo):

```
sudo setcap cap_net_raw+eip "$(readlink -f .venv/bin/python3)"
```

Caveat: `.venv/bin/python3` is a symlink; the capability lands on the **real** interpreter it points to. If that's the system `/usr/bin/python3.13`, every venv on the host built from it can open raw sockets.

Acceptable on a personal machine; on shared hosts build the venv with a private interpreter first (`uv venv --python 3.13` uses a uv-managed copy under `~/.local/share/uv/python/`). Revoke anytime:

```
sudo setcap -r "$(readlink -f .venv/bin/python3)"
```

C. systemd service (continuous monitoring, survives reboots) — see section 7.

4. Run

```
uv run netmon.py                # all interfaces incl. loopback/  
                                VPN-tun/docker  
uv run netmon.py -q             # files only (recommended for long runs)  
uv run netmon.py -i wlan0       # single interface  
uv run netmon.py --bpf 'not port 22' # exclude your own SSH session  
                                noise  
uv run netmon.py -o /var/log/netmon # custom output root (default: ./  
                                logs)
```

Each start creates a fresh `logs/run-YYYYMMDD-HHMMSS/`. Stop with `Ctrl-C` (or `kill -TERM <pid>`) — the summary is only written on clean shutdown.

Startup prints `capture_started` with the interface list and detected local IPs — check that list; direction (inbound/outbound) classification depends on it. A `stats` line is logged every 30 s with packets, event counts, queue depth, `userspace_dropped` (netmon's own queue overflowed) and `kernel_dropped` (the kernel's `tp_drops`: packets the socket buffer shed before netmon ever saw them). `kernel_dropped: "unavailable"` means the source can't report it (e.g. pcap replay).

`--read file.pcap` replays a capture file through the same pipeline instead of sniffing live — no privileges needed; useful for testing and offline analysis.

5. Verify capture is working

From a second terminal, generate one known event of each type:

```
dig example.com # → dns_query + dns_answer in dns.jsonl
curl -s https://example.com >/dev/null # → tls_sni in tls.jsonl, flow in flows.jsonl
curl -s http://neverssl.com >/dev/null # → http event in http.jsonl
```

Then confirm:

```
ls logs/run-*/ # dns.jsonl tls.jsonl http.jsonl
flows.jsonl
jq -r 'select(.sni=="example.com")' logs/run-*/tls.jsonl
```

If those appear, capture is healthy.

6. Output reference

File	One line per	Key fields
<code>dns.jsonl</code>	DNS query / answer / HTTPS-SVCB record	<code>qname</code> , <code>qtype</code> , <code>rtype</code> , <code>value</code> , <code>ttl</code> , <code>resolver</code> ; HTTPS/SVCB (type 65/64) adds <code>alpn</code> , <code>port</code> , <code>ipv4hint</code> , <code>ipv6hint</code> , <code>ech</code> , <code>target</code>
<code>tls.jsonl</code>	ClientHello with SNI (TCP TLS or QUIC)	<code>sni</code> , <code>dst</code> , <code>dport</code> , <code>transport</code> (<code>tcp</code> / <code>quic</code>), <code>alpn</code> (negotiated protocols), <code>ech</code> (<code>true</code> = <code>sni</code> is an ECH cover name)
<code>http.jsonl</code>	plaintext HTTP request	<code>method</code> , <code>host</code> , <code>path</code> , <code>user_agent</code> , <code>tag</code> (e.g. <code>captive-portal</code>)

<code>flows.jsonl</code>	new 5-tuple connection	<code>direction</code> , <code>scope</code> (<code>internet</code> / <code>lan</code> / <code>multicast</code>), <code>service</code> , <code>hostname</code> , <code>remote_ip</code> , <code>note</code> (on NTP / STARTTLS-mail flows)
<code>summary.json</code>	run (written at exit)	top DNS/SNI names, top internet hosts, event counts, <code>capture</code> drop counters (<code>userspace_dropped</code> , <code>kernel_dropped</code>)

All events: ISO 8601 UTC `ts` with millisecond precision. `hostname` on flows is reverse-mapped from DNS answers observed **during this run** — flows to IPs resolved before the monitor started show no hostname.

Analysis recipes

```
R=logs/run-20260703-090000 # pick a run

jq -r '.sni' $R/tls.jsonl | sort | uniq -c | sort -rn #
sites visited (SNI)
jq -r 'select(.kind=="dns_query").qname' $R/dns.jsonl | sort -u #
every name resolved
jq -r 'select(.scope=="internet") | .hostname // .remote_ip' $R/
flows.jsonl \
| sort | uniq -c | sort -rn #
internet endpoints
jq -rs 'sort_by(.ts) | .[] | "\(.ts) \(.kind) \
(.sni // .qname // .hostname // .remote_ip // "")"' \
$R/*.jsonl #
merged timeline
```

What your ISP sees even with HTTPS: every `qname` in `dns.jsonl` (unless DNS is encrypted), every `sni` in `tls.jsonl`, every `remote_ip` in internet-scope flows, and everything in `http.jsonl` in full. And beyond what netmon can capture — traffic-analysis (packet sizes/timing), TLS fingerprints, TLS 1.2 server certs, IPv6 MAC leakage — see “What this tool does NOT show you” in the [README](#).

7. Continuous monitoring (systemd)

`/etc/systemd/system/netmon.service` :

```
[Unit]
Description=netmon passive network monitor
After=network.target

[Service]
WorkingDirectory=/opt/netmon
ExecStart=/opt/netmon/.venv/bin/python netmon.py -q -o /var/log/netmon
AmbientCapabilities=CAP_NET_RAW
CapabilityBoundingSet=CAP_NET_RAW
```

```
NoNewPrivileges=true
ProtectSystem=strict
ReadWritePaths=/var/log/netmon
Restart=on-failure
KillSignal=SIGTERM
```

```
[Install]
WantedBy=multi-user.target
```

```
sudo mkdir -p /var/log/netmon
sudo systemctl daemon-reload
sudo systemctl enable --now netmon
journalctl -u netmon -f # structlog JSON: capture_started,
                        stats, capture_stopped
```

`AmbientCapabilities` makes `setcap` (section 3B) unnecessary for the service. Each restart opens a new run directory, which doubles as log rotation. Prune old runs:

```
find /var/log/netmon -maxdepth 1 -name 'run-*' -mtime +30 -exec rm -r
{} +
```

8. Troubleshooting

Symptom	Cause / fix
<code>insufficient_privileges</code> at startup	No root/CAP_NET_RAW — section 3. After rebuilding <code>.</code> , <code>venv</code> (e.g. <code>uv sync</code> upgraded Python), <code>setcap</code> must be re-applied.
<code>sudo: uv: command not found</code>	Use <code>sudo \$(command -v uv) run netmon.py</code> .
Starts, but zero packets in <code>stats</code>	Wrong <code>-i</code> name (check <code>ip link</code>); or a too-narrow <code>--bpf</code> expression.
<code>--bpf</code> raises an error	<code>tcpdump</code> not installed — <code>scapy</code> needs it to compile the filter.
<code>userspace_dropped</code> > 0 in stats	Consumer can't keep up (very busy link). Narrow with <code>--bpf</code> or <code>-i</code> , and use <code>-q</code> .
<code>kernel_dropped</code> > 0 in stats	The kernel shed packets before netmon read them — the run under-reports by at least that many packets; treat the record as incomplete. Same remedies: <code>--bpf</code> , <code>-i</code> , <code>-q</code> .
Flows show <code>direction: "transit"</code>	Neither endpoint is a local IP — broadcast/multicast from other LAN devices, or you're capturing a mirrored/bridged port. Normal.
Flow has no <code>hostname</code>	The DNS lookup happened before this run started, or the app used DoH. Correlate <code>remote_ip</code> manually.
DNS queries only show <code>dst: 127.0.0.53</code>	<code>systemd-resolved</code> stub on loopback — those are your apps' real queries. If no matching upstream <code>udp/53</code> appears on a physical interface, the upstream leg is inside a VPN tunnel (e.g. Tailscale MagicDNS): your ISP does not see that DNS.

Nothing in <code>http.jsonl</code>	Expected — nearly everything is HTTPS now. It only catches plaintext HTTP.
No SNI for QUIC/HTTP-3 flows (<code>service: "quic"</code>)	netmon decrypts <code>udp/443</code> client Initials and logs the SNI to <code>tls.jsonl</code> with <code>transport: "quic"</code> . A QUIC flow with no SNI means QUIC on a non-443 port, ECH in use, or the Initial wasn't captured.
No SNI for some TCP HTTPS traffic	Encrypted Client Hello (ECH) genuinely hides it — a leak prevented, not a gap; see README limitations.

9. Ops notes

- Purely passive: sends nothing, modifies nothing; promiscuous mode is off (you see this host's traffic plus broadcast/multicast).
- The logs themselves are the sensitive artifact — they are your browsing history. Each run directory is created owner-only (`0700`) and every `*.jsonl` / `summary.json` is `0600`, so other local users can't read captured traffic. The **output root** (e.g. `logs/`, or a `-o` path) keeps whatever mode it already had — if it predates this and is world-listable, `chmod 700` it yourself so run-directory timestamps aren't enumerable. Still: prune old runs and don't sync them to shared storage.
- Memory is bounded — every per-key structure evicts: the flow-dedup table is an LRU capped at 200k five-tuples (an idle flow can age out and re-emit one `flow` event; an active one never does), the IP→hostname ledger is an LRU capped at 64k entries, and the DNS/SNI/host tallies cap at 50k keys each (past that, only the hottest keys keep exact counts, so `top_*` lists stay right but `unique_*` totals become estimates). Worst case is on the order of 100 MB of tables on a pathological link, tens of MB in practice. JSONL files are flushed per line, so a crash loses at most the final summary (re-derivable from the JSONL with `jq`).